PBScaler: A Bottleneck-Aware Autoscaling Framework for Microservice-Based Applications

Shuaiyu Xie, Jian Wang, *Member, IEEE*, Bing Li, *Member, IEEE*, Zekun Zhang, Duantengchuan Li, and Patrick C. K. Hung

***Abstract*—Autoscaling is critical for ensuring optimal performance and resource utilization in cloud applications with dynamic workloads. However, traditional autoscaling technologies are typically no longer applicable in microservice-based applications due to the diverse workload patterns and complex interactions between microservices. Specifically, the propagation of performance anomalies through interactions leads to a high number of abnormal microservices, making it difficult to identify the root performance bottlenecks (PBs) and formulate appropriate scaling strategies. In addition, to balance resource consumption and performance, the existing mainstream approaches based on online optimization algorithms require multiple iterations, leading to oscillation and elevating the likelihood of performance degradation. To tackle these issues, we propose PBScaler, a bottleneck-aware autoscaling framework designed to prevent performance degradation in a microservice-based application. The key insight of PBScaler is to locate the PBs. Thus, we propose TopoRank, a novel random walk algorithm based on the topological potential to reduce unnecessary scaling. By integrating TopoRank with an offline performance-aware optimization algorithm, PBScaler optimizes replica management without disrupting the online application. Comprehensive experiments demonstrate that PBScaler outperforms existing state-of-the-art approaches in mitigating performance issues while conserving resources efficiently.**

***Index Terms*—Microservice, autoscaling, performance bottleneck, replica management.**

I. Introduction

With the advancement of microservice architecture, an increasing number of cloud applications are migrating from monolithic architecture to microservice architecture [1], [2], [3], [4], [5], [6]. This new architecture reduces application coupling by breaking a monolithic application into multiple microservices that communicate with each other via HTTP or RPC protocols [7]. Moreover, each microservice can be developed, deployed, and scaled independently by separate teams, enabling rapid application development and iteration. Nevertheless, the unpredictability of external workloads and the complexity of interactions between microservices can result in performance degradation [8], [9], [10]. Cloud providers must prepare excessive resources to meet the service level objective (SLO) of application owners, which usually causes unnecessary waste of resources [11], [12]. As a result, the imbalance between satisfying SLO and minimizing resource consumption becomes a major challenge encountered by resource management in microservices.

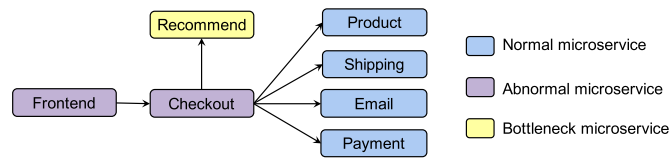


Fig. 1. Part of the invocation relationship in Online Boutique.

Microservice autoscaling refers to the capability of allocating resources elastically in response to workload variations [13]. By utilizing the elasticity property of microservices, autoscaling can mitigate the conflict between resource cost and performance. However, the autoscaling of microservices suffers from accurately scaling the performance bottleneck (PB) in a short period. Due to the complexity of communication between microservices, the performance degradation of a PB may propagate to other microservices via message passing [2], resulting in a high number of abnormal microservices at the same time. We demonstrated this by injecting burst workloads to specific microservices in Online Boutique,[1] an open-source microservice application developed by Google. Fig. 1 shows that the performance degradation in the PB *Recommend* can spread to the upstream microservices like *Checkout* and *Frontend*. To further verify the importance of accurately scaling the PB, we conducted stress testing and scaled different microservices separately. As shown in Fig. 2, abnormal microservice (*Frontend*) scaling cannot alleviate SLO violations. However, when we identified and scaled the PB *Recommend*, the performance of the microservice application improved. Unfortunately, locating PBs is usually time-consuming and can occasionally make mistakes [14].

In recent years, several approaches have been proposed to identify critical microservices before autoscaling. For example,

Manuscript received 16 March 2023; revised 25 December 2023; accepted 26 February 2024. Date of publication 18 March 2024; date of current version 10 April 2024. This work was supported in part by the National Key Research and Development Program of China under Grant 2022YFF0902701 and in part by the National Natural Science Foundation of China under Grant 62032016, Grant 61832014, and Grant 61972292. *(Corresponding authors: Jian Wang; Bing Li.)*

Shuaiyu Xie, Jian Wang, Bing Li, Zekun Zhang, and Duantengchuan Li are with the School of Computer Science, Wuhan University, Wuhan 430072, China (e-mail: theory@whu.edu.cn; jianwang@whu.edu.cn; bingli@whu.edu.cn; zekunzhang@whu.edu.cn; dtclee1222@whu.edu.cn).

Patrick C. K. Hung is with the Faculty of Business and Information Technology, Ontario Tech University, Oshawa, ON L1G 0C5, Canada (e-mail: patrick.hung@ontariotechu.ca).

Digital Object Identifier 10.1109/TSC.2024.3376202

[1]https://github.com/GoogleCloudPlatform/microservices-demo

1939-1374 © 2024 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: University of Greenwich. Downloaded on April 07,2026 at 15:55:01 UTC from IEEE Xplore. Restrictions apply.

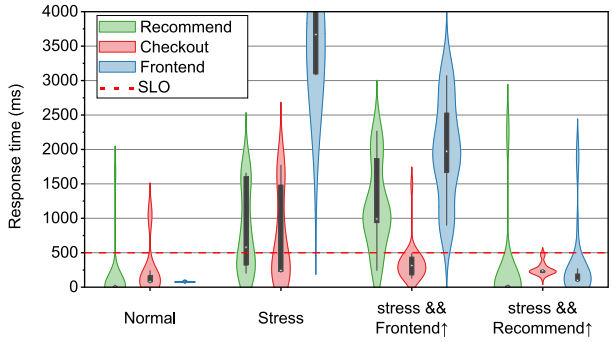


Fig. 2. Latency distribution in four scenarios.

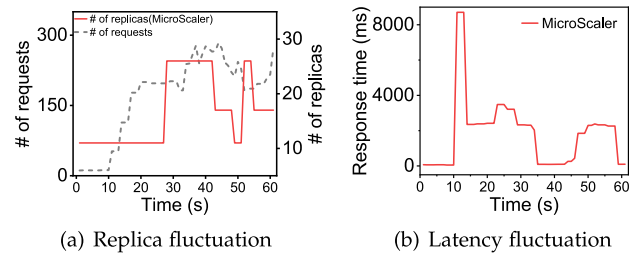


Fig. 3. Replica fluctuation and latency fluctuation of MicroScaler under burst workloads. Excessive online attempts (a) cause oscillations and performance degradation (b).

the default autoscaler of Kubernetes[2] filters microservices for direct scaling based on a static threshold of computing resources. Yu et al. [15] defined the boundaries of elastic scaling by calculating the service power, which is the ratio between the 50th percentile response time (P50) and the 90th percentile response time (P90). Furthermore, Qiu et al. [4] introduced an SVM-based approach for extracting critical paths by analyzing the ratio of various tail latencies. Although these studies have narrowed the scope of autoscaling, they still take into account non-bottleneck microservices that may affect scaling strategies, especially when a large number of microservices in the application are abnormal at the same time. Consequently, there is an urgent need to pinpoint bottleneck microservices accurately before autoscaling.

To balance resource consumption and performance, existing works have employed online optimization algorithms to find near-optimal autoscaling strategies. However, due to the vast range of possible strategies for autoscaling, these approaches require a significant number of attempts, which will be problematic for online applications. For example, Train Ticket[3] is the largest open-source microservice application, consisting of nearly 40 microservices. Assuming that each microservice can have up to 15 replicas, determining the optimal allocation strategy for this application is undoubtedly an NP-hard problem, as there are a maximum of 15^{40} scaling alternatives. Additionally, the duration of the feedback loop in online optimization is too long to achieve model convergence. It is also essential to consider the potential risks of performance degradation caused by online optimization. Fig. 3 illustrates the impact of burst workloads on the replica fluctuation and latency fluctuation of MicroScaler [15], an online autoscaling approach incorporating online Bayesian optimization to find the global minimizer of the total cost. The frequent online attempts to create replicas (Fig. 3(a)) caused by online optimization result in oscillations and performance degradation (Fig. 3(b)). As a result, we are inspired to design an offline optimization process fueled by feedback from a simulator.

This paper presents PBScaler, a horizontal autoscaling framework designed to prevent performance degradation in microservice-based applications by identifying and addressing bottlenecks. Instead of optimizing resources for all abnormal microservices, as was done in previous work [11], [15], we propose TopoRank, a random walk algorithm based on topological potential theory (TPT) to identify performance bottlenecks (PBs). By taking into account microservice dependencies and anomaly potential, TopoRank improves the accuracy and explainability of bottleneck localization. After identifying the PBs by TopoRank, PBScaler further employs a genetic algorithm to find nearly optimal strategies. To avoid application oscillation caused by excessive optimization, the process is conducted offline and is guided by an SLO violation predictor, which simulates the online application and provides feedback to the scaling strategies. The main contributions of the paper are summarized as follows:

- We propose PBScaler, a bottleneck-aware autoscaling framework designed to prevent performance degradation in a microservice-based application. By pinpointing bottlenecks, PBScaler can reduce unnecessary scaling and expedite the optimization process.
- We employ a genetic algorithm-based offline optimization process to optimize resource consumption while avoiding SLO violations. This process is guided by an SLO violation predictor and is designed to strike a balance between resource consumption and performance without disrupting online applications.
- We design and implement PBScaler in the Kubernetes system. To evaluate its effectiveness, we conduct extensive experiments with real-world and emulated workload injection on two widely-used microservice systems running in an online environment. Experimental results demonstrate that PBScaler outperforms several state-of-the-art elastic scaling methods.

The rest of the paper is organized as follows. In Section II, we discuss the related work about bottleneck analysis and autoscaling for microservices. In Section III, we describe the overall system in detail. In Section IV, we present the evaluations and experimental results. Section V concludes our work and discusses the future research direction.

II. Related Work

With the advancement of cloud computing, numerous autoscaling methods for cloud resources, such as virtual machines or containers, have been proposed in academia and industry [16], [17], [18], [19]. However, autoscaling for microservices can be much more complicated due to the intricate dependencies between microservices.

[2]https://github.com/kubernetes/autoscaler
[3]https://github.com/FudanSELab/train-ticket

Authorized licensed use limited to: University of Greenwich. Downloaded on April 07,2026 at 15:55:01 UTC from IEEE Xplore. Restrictions apply.

Performance bottleneck analysis, also known as root cause analysis, is a useful way for quickly locating the bottleneck responsible for the performance degradation of microservices, hence reducing the time and effort required for autoscaling. In this section, we will analyze the related work on bottleneck analysis and autoscaling for microservices.

A. Bottleneck Analysis

In recent years, numerous methods for bottleneck analysis in microservice scenarios have been developed, most of which rely on three types of data: logs, traces, and metrics. 1) Logs. Jia et al. [20] and Nandi et al. [21] first extracted templates and flows from normal-state logs, matched them with target logs, and filtered out abnormal logs. 2) Traces. Trace is an event tracking-based record that reproduces the request process between microservices. Several studies [22], [23], [24], [25] have been introduced to pinpoint the bottlenecks using traces. Yu et al. [22], [23] located bottlenecks by combining spectrum analysis and the PageRank algorithm on the dependency graph constructed by traces, while Mi et al. [24] presented an unsupervised machine learning prototype to learn the pattern of microservices and filter out abnormal microservices. However, using traces can be intrusive to the code and requires operators to have a deep understanding of the structure of the microservices. 3) Metrics. Some approaches [2], [26], [27] leverage graph random walk algorithms to simulate the propagation process of anomalies and then find bottlenecks by integrating statistical features of metrics and dependencies between microservices. Additionally, methods such as CauseInfer [14] and MicroCause [28] focused on building metrics causality graphs with causal inference, which typically involve hidden indirect relationships between metrics.

Since the workflow code is rarely modified when monitoring metrics, collecting metrics for microservices is usually cheaper than using trace. Moreover, using metrics as the primary monitoring data can reduce the cost of integrating bottleneck analysis and autoscaling, as metrics are widely used in the latter scenario. Despite these approaches' advantages, most have no preference in selecting the starting point for abnormal backtracking. In contrast, our approach begins random walks from microservices with greater anomaly potential, accelerating convergence speed and improving bottleneck localization accuracy.

B. Autoscaling for Microservices

Existing autoscaling methods for microservices can be categorized into five groups. 1) Rule-based heuristic approach. KHPA, Libra [29], KHPA-A [30], and PEMA [31] manage the number of microservice replicas based on resource thresholds and specific rules. However, since different microservices have varying sensitivities to specific resources, expert knowledge is needed to support autoscaling for these various microservices. 2) Model-based approach. Microservices can be modeled to predict their status under particular configurations and workloads. Queuing theory [32], [33] and graph neural network (GNN) [12] are commonly used to build performance prediction models for microservices. 3) Control theory-based approach [11], [32]. Using the control theory, SHOWAR [11] dynamically adjusts the microservice replicas to correct the error between monitoring metrics and thresholds. 4) Optimization-based approach. These methods [15], [34] make a large number of attempts to find the optimal strategy given the present resources and workloads. The key to these approaches is to reduce the decision-making scope to speed up the process. 5) RL-based Approach. Reinforcement learning (RL) has been widely used in resource management for microservices. MIRAS [35] adopts a model-based RL method for decision-making to avoid the high sampling complexity of the real environment. FIRM [4] leverages a support vector machine (SVM) to identify the critical path in microservices and a deep deterministic policy gradient (DDPG) algorithm to make hybrid scaling strategies for microservices along the path. RL-based methods require constant interactions with the environment during exploration and are incapable of adapting to the dynamic microservices architecture.

In conclusion, while the aforementioned autoscaling techniques have their respective advantages, they pay little attention to performance bottlenecks. Consuming computer resources for non-bottleneck microservices will inevitably increase scaling costs and lengthen decision-making. Our method, on the other hand, focuses on locating performance bottlenecks.

III. System Design

We present PBScaler, a PB-centric autoscaling controller, to locate PBs and optimize replicas for them. As shown in Fig. 4, PBScaler comprises three components: 1) *Metric Collector:* To provide real-time insights into the applications' status, we design a metric collector that captures and integrates monitoring metrics from Prometheus[4] at fixed intervals. 2) *Performance Bottleneck Analysis:* With the assistance of the *metric collector*, this component performs SLO violation detection and redundancy checking to identify microservices with abnormal behavior. Next, the bottleneck localization process will be triggered to pinpoint the PBs in the abnormal microservices. 3) *Scaling Decision:* This component aims to determine the optimal number of replicas for PBs using an evolutionary algorithm. Finally, PBScaler generates configuration files with optimized strategies and commits them to the kubernetes-client,[5] which regulates the replica count of microservices.

A. Metric Collector

The autoscaling controller relies on observability for microservice applications, such as system load, tail latency, and invocation relationships between microservices, to determine whether elastic scaling should be performed and how many resources should be allocated. While a trace-based monitor can reflect the dependencies and performance of microservices, it requires a deep understanding of the program and code injection [7]. Moreover, real-time analysis of massive tree-structural traces demands a considerable amount of processing time. Therefore, we design a metric-based monitor, the

[4]https://prometheus.io
[5]https://github.com/kubernetes-client/python

Authorized licensed use limited to: University of Greenwich. Downloaded on April 07,2026 at 15:55:01 UTC from IEEE Xplore. Restrictions apply.

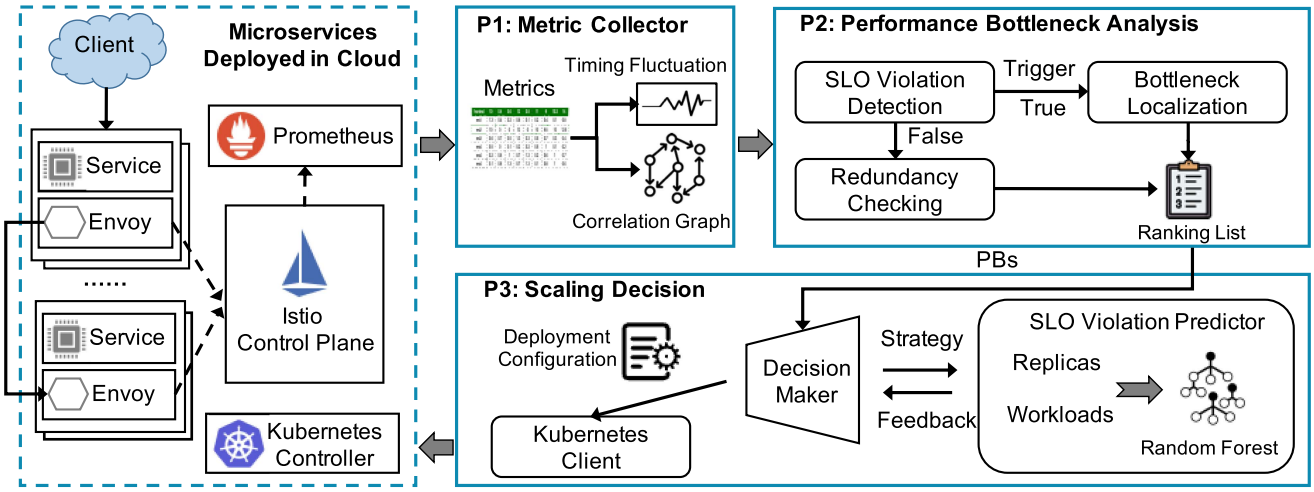


Fig. 4. Framework of PBScaler.

TABLE I
LABELS OF METRICS COLLECTED FROM THE MONITORING TOOLS

Level	Label
Container Level	kube_pod_info container_cpu_usage_seconds_total container_memory_usage_bytes container_spec_cpu_quota container_spec_memory_limit_bytes container_fs_usage_bytes container_fs_write_seconds_total container_fs_read_seconds_total container_network_receive_bytes_total container_network_transmit_bytes_total
Microservice Level	istio_request_duration_milliseconds_bucket istio_requests_total istio_tcp_received_bytes_total

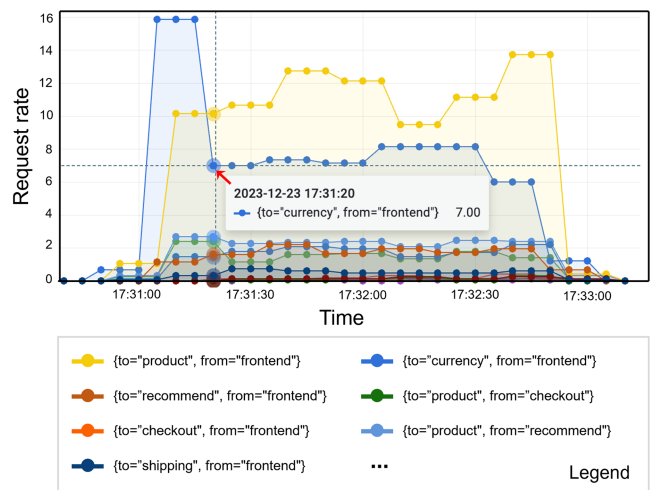


Fig. 5. Visualization of query results for metric *istio_requests_total*.

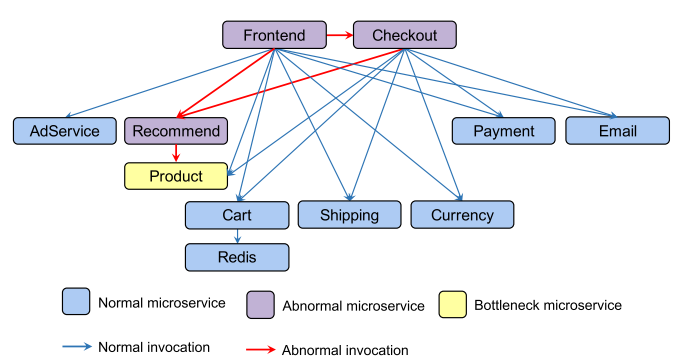


Fig. 6. Example of anomaly propagation in microservices.

Metric Collector, based on non-intrusive service mesh technology to minimize disruptions to business flows. As shown in Table I, PBScaler uses Prometheus and kube-state-metrics to gather and categorize these metrics (M), including tail latency, invocation relationships between microservices, resource consumption, and microservice workload. For example, *container_cpu_usage_seconds_total* is a resource metric that records the Central Processing Unit (CPU) usage at the container level. *istio_requests_total* records the TCP requests between microservices. Fig. 5 displays a visualization of the query results for *istio_requests_total*, where each point represents the average request rate within the preceding minute for a specific invocation relationship (e.g., *frontend*→*currency* as marked in Fig. 5). This information serves as a means to intuitively reflect the workload on microservices. Furthermore, *istio_requests_total* also records all invocation relationships (listed in the legend of Fig. 5), which can be utilized to construct a microservice correlation graph $\mathcal{G}_c$ similar to the one depicted in Fig. 6. Meanwhile, *istio_request_duration_milliseconds_bucket* serves as a performance metric reflecting the tail latency of a microservice. Typically, we collect the P90 tail latency of each microservice to observe their performance. The monitoring interval of Prometheus is set to five seconds, and the collected metrics data is stored in a time-series database.

Service Mesh. A service mesh is an infrastructure that enables developers to add advanced features, such as observability and

Authorized licensed use limited to: University of Greenwich. Downloaded on April 07,2026 at 15:55:01 UTC from IEEE Xplore. Restrictions apply.

traffic management, to cloud applications without requiring additional code. One popular open-source service mesh implementation is Istio,[6] designed to seamlessly integrate with Kubernetes. When a pod starts up in Kubernetes, Istio launches an envoy proxy within the pod to intercept network traffic, enabling workload balancing and monitoring.

B. Performance Bottleneck Analysis

Performance Bottleneck Analysis (PBA) is a process designed to discover performance degradation and resource waste in microservice applications to infer PBs of the current problem. As stated in Section I, by accurately locating these bottlenecks, PBA can enhance the performance of autoscaling and reduce excessive resource consumption. The PBA process in PBScaler is depicted in Algorithm 1.

1) SLO Violation Detection: To detect abnormalities in microservices, PBScaler uses service level objectives (SLOs) to compare with specific metrics. It is considered abnormal if a microservice has numerous SLO violations, i.e., performance degradation. As discussed in [14], [27], detecting SLO violations is a critical step in triggering bottleneck localization. The invocation relationships collected by the *Metric Collector* can be leveraged to build a microservice correlation graph $\mathcal{G}_c$. PBScaler inspects the P90 tail latency of all invocation edges in $\mathcal{G}_c$ every 15 seconds to timely detect performance degradation. If the tail latency of an invocation exceeds a predetermined threshold (such as the SLO), the invoked microservice of the invocation will be added to the set of abnormal microservices ($\mathbb{S}$), and the bottleneck localization process will be activated. To account for occasional noise in the microservice latency, the threshold is set to SLO$\times(1+\frac{\alpha}{2})$, where α is used to adjust the tolerance to noise.

2) Redundancy Checking: In the absence of performance anomalies, some microservices may be allocated more resources than required. However, identifying such cases can be difficult through metrics alone, potentially leading to wasting limited hardware resources. To avoid this, it is essential to identify which microservices have allocated excess resources. PBScaler uses the rate of workload change per second of microservices to determine whether resources are redundant. This strategy is more effective than relying only on resource consumption because different microservices may have varying sensitivity to heterogeneous resources. The main idea behind redundancy checking is to employ hypothesis testing to detect whether a microservice's current workload $\boldsymbol{w_c^i}$ is significantly lower than its past workload (denoted as $\boldsymbol{w_p^i}$). The degree of significance is adjusted by the parameter β, and the hypothesis test can be formulated as

$$\begin{cases} H_0, & \boldsymbol{w_c^i} \geq \boldsymbol{w_p^i} \times \beta \\ H_1, & \boldsymbol{w_c^i} < \boldsymbol{w_p^i} \times \beta. \end{cases} \tag{1}$$

To perform the hypothesis test, we first fetch the current and historical workloads of the target microservices from the *Metric Collector*. We then use a one-sided test to compute the p-value

[6]https://istio.io

Algorithm 1: Performance Bottleneck Analysis.

Input: SLO
microservice correlation graph $\mathcal{G}_c$
confidence level cl
tolerance to noise α
degree of significance β
impact factor σ
Output: ranking list $\boldsymbol{rl}$
Initialize $\boldsymbol{rl}$ and abnormal microservice set $\mathbb{S}$
for each v_i in $\mathcal{G}_c$.nodes **do**
  for each $e_{j,i}$ in in-edges of v_i **do**
    /* SLO violation detection */
    if $P90(e_{j,i}) >$ SLO $*(1+\alpha/2)$ **then**
      Store v_i in $\mathbb{S}$
    end if
  end for
end for
if $\mathbb{S}$ is empty **then**
  /* Redundancy checking */
  for each v_i in $\mathcal{G}_c.nodes$ **do**
    $\boldsymbol{w_p^i}, \boldsymbol{w_c^i} \leftarrow$ Get workloads for microservice i from the Metric Collector
    $t, P \leftarrow ttest(\boldsymbol{w_c^i}, \boldsymbol{w_p^i} * \beta)$
    if $t < 0$ and $P < cl$ **then**
      Store v_i in $\boldsymbol{rl}$
    end if
    Return $\boldsymbol{rl}$
  end for
end if
$\boldsymbol{rl} = TopoRank(\mathbb{S})$
Return $\boldsymbol{rl}$

P. If P does not exceed the confidence level cl (which is set to 0.05 by default), we reject the null hypothesis H_0 and consider the microservice i to have redundant resources.

3) Bottleneck Localization: Because of the complex interactions in a microservice application [36], not every abnormal microservice needs to be scaled up. For example, Fig. 6 illustrates how the performance degradation of a bottleneck microservice (e.g., *Product*) can propagate to its upstream microservices (e.g., *Recommend*, *Frontend*, and *Checkout*) along the invocation chains, even if the upstream microservices are not overloaded. Therefore, only the bottleneck microservice must be scaled up while the other abnormal microservices are merely implicated. To pinpoint the bottleneck microservice, we introduce the concept of anomaly potential, which aggregates the anomaly impact of all microservices in a given position. Generally, a performance bottleneck tends to exhibit a high anomaly potential in a microservice application, as it is frequently surrounded by abnormal neighbors that are influenced by the bottleneck. Based on the anomaly potential, we model the backtracking of anomalies in microservice applications as a random walk on a directed graph. Therefore, We design a novel bottleneck localization algorithm, TopoRank, which introduces the topological potential theory (TPT) in random walks to calculate the scores

Authorized licensed use limited to: University of Greenwich. Downloaded on April 07,2026 at 15:55:01 UTC from IEEE Xplore. Restrictions apply.

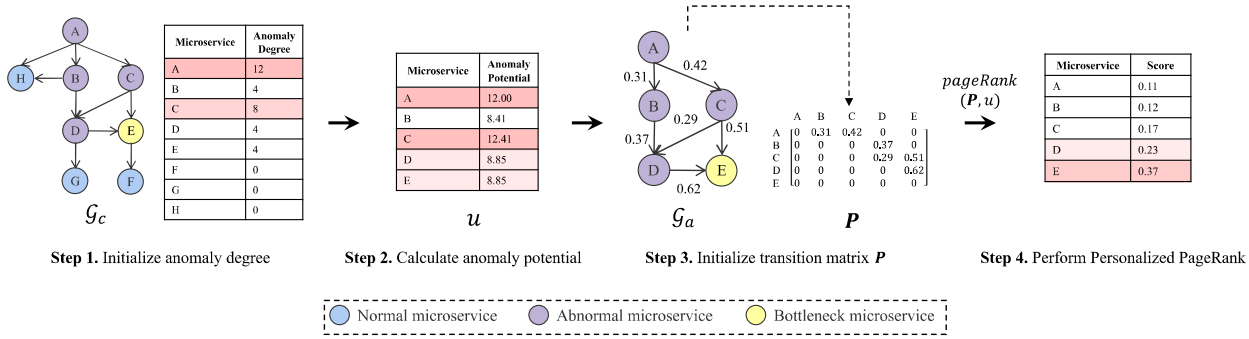


Fig. 7. Illustration of the computation process for the TopoRank algorithm.

Algorithm 2: TopoRank.

Input: abnormal subgraph $\mathcal{G}_a$
impact factor σ
preference vector $\boldsymbol{u}$
transition matrix $\boldsymbol{P}$
collected metrics M
Output: ranking list $\boldsymbol{rl}$

```
Initialize rl and preference vector u
for each v_i in G_a.nodes do
    a_i ← Anomaly degree of v_i
    φ ← a_i
    for each v_j in upstream microservices of v_i do
        h_ji ← Minimum number of hops from v_j to v_i
        a_j ← Anomaly degree of v_j
        φ ← φ + a_j e^{-(h_ji/σ)^2}
    end for
    u_i ← φ
    L_t ← An array of tail latency for v_i
    for each v_j in out-edges of v_i do
        L_m ← An array of metric m for v_j
        P_{i,j} ← max_{m∈M}(corr(L_t, L_m))
    end for
end for
rl ← pageRank(P, u)
Return rl
```

for all abnormal microservices and finally outputs a ranking list ($\boldsymbol{rl}$). The microservices with the highest scores in the $\boldsymbol{rl}$ can be recognized as the PBs.

TPT, which originated from the concept of "field" in physics, has been widely used in various works [37], [38] to measure the mutual influence between nodes in complex networks. Since the microservice correlation graph can also be viewed as a complex network, we use TPT to evaluate the anomaly potential of microservices. Specifically, we have observed that in a microservice correlation graph $\mathcal{G}_c$, the microservices closer to the PBs, i.e., those with fewer hops, are more likely to be abnormal, as they often have frequent direct or indirect invocations with the PBs. Based on this observation, we evaluate the anomaly potential of microservices using the TPT. To do this, we first extract the abnormal subgraph $\mathcal{G}_a$ by identifying the abnormal microservices and their invocation relationships in $\mathcal{G}_c$. We then calculate the anomaly potential φ for microservice v_i in the abnormal subgraph $\mathcal{G}_a$ using the TPT

$$\varphi = a_i + \sum_{j=1}^{N} a_j e^{-(h_{ji}/\sigma)^2}, \tag{2}$$

where N is the number of upstream microservices of v_i and a_j represents the anomaly degree of v_j. PBScaler defines the anomaly degree as the number of SLO violations for a microservice in a time window. h_{ji} denotes the minimum number of hops required from v_j to v_i. We use the impact factor σ (1 by default) to control the influence range of a microservice.

Fig. 7 illustrates the computation process of the TopoRank algorithm. PBScaler checks the number of SLO violations for each microservice in Step 1 to initialize the anomaly degree. It can be observed that the microservices A and C, positioned upstream of the performance bottleneck (i.e., microservice E), have the highest anomaly degrees, with values of 12 and 8, respectively. This is because anomalies in downstream microservices propagate and accumulate in upstream microservices. Consequently, it is inadvisable to locate PBs solely based on the anomaly degree. In Step 2, PBScaler calculates the anomaly potential for each microservice according to (2). Taking the microservice C as an example, its anomaly potential φ can be calculated as: $8 + 12 \times e^{-(1/1)^2} = 12.41$.

However, microservices with high anomaly potential values are not necessarily PBs, since anomalies are usually propagated along the microservice correlation graph. In Step 2 of Fig. 7, the upstream microservice A exhibits the highest anomaly potential due to the propagation of downstream anomalies. Hence, relying solely on the TPT is insufficient for diagnosing PBs. To address this issue, PBScaler incorporates the Personalized PageRank algorithm [39] to reverse the anomaly propagation on the abnormal subgraph $\mathcal{G}_a$ and locate PBs. Let $\boldsymbol{P}$ be the transition matrix of $\mathcal{G}_a$ and $\boldsymbol{P}_{i,j}$ be the probability of anomaly tracking from v_i to its downstream node v_j. Given v_i with out-degree d, the standard Personalized PageRank algorithm sets $\boldsymbol{P}_{i,j}$ as

$$\boldsymbol{P}_{i,j} = \frac{1}{d}, \tag{3}$$

Authorized licensed use limited to: University of Greenwich. Downloaded on April 07,2026 at 15:55:01 UTC from IEEE Xplore. Restrictions apply.

which means that the algorithm is not biased toward any downstream microservice. However, this definition fails to consider the association between the downstream microservices and the anomaly of the current microservice. Consequently, PBScaler adapts the calculation by giving more attention to the downstream microservices whose metrics are more relevant to upstream response time. For each microservice v_i, PBScaler logs a tail latency array (L_t). For any metric m (e.g., $container_spec_cpu_quota$) of collected metrics M in Table I, PBScaler records a metric array L_m for v_i within a specified time window. PBScaler defines that $\boldsymbol{P}_{i,j}$ depends on the maximum value of the Pearson correlation coefficient between L_t and L_m

$$\boldsymbol{P}_{i,j} = \max_{m \in M}(corr(L_t, L_m)). \tag{4}$$

The Personalized PageRank algorithm determines the popularity of each node by randomly walking on the directed graph. However, some nodes may never point to others, causing the scores of all nodes to tend toward zero after many iterations of the random walk. To avoid falling into this "trap," a damping factor δ is applied, which allows the algorithm to jump out from these nodes according to a predefined rule. Typically δ is set to 0.15. The Personalized PageRank is represented as follows:

$$\boldsymbol{v} = (1-\delta) \cdot \boldsymbol{P}\boldsymbol{v} + \delta \cdot \boldsymbol{u}, \tag{5}$$

where $\boldsymbol{v}$ represents the probability that each microservice node is diagnosed as a PB. The preference vector $\boldsymbol{u}$ serves as the personalized rule to guide the algorithm to leap from the trap. The value of $\boldsymbol{u}$ is determined by the anomaly potential φ of each node. The nodes with greater anomaly potential are preferred as starting points for the algorithm. The equation of the kth iteration can be represented as

$$\boldsymbol{v}^{(k)} = (1-\delta) \cdot \boldsymbol{P}\boldsymbol{v}^{(k-1)} + \delta \cdot \boldsymbol{u}. \tag{6}$$

After multiple rounds of iterations, $\boldsymbol{v}$ gradually converges. PBScaler then sorts the final results and produces the ranking list $\boldsymbol{rl}$. In Step 4 of Fig. 7, the TopoRank algorithm undergoes multiple rounds of random walks. During this process, it reduces the suspicion levels on microservices A and C, assigning them low scores of 0.11 and 0.17, respectively. Conversely, microservice E is identified with the highest suspicion score of 0.37. The top-k microservices (E and D) with the highest ranking list scores can be recognized as PBs. The whole process of TopoRank is depicted as Algorithm 2.

C. Scaling Decision

Given the PBs identified by the *Performance Bottleneck Analysis*, the replicas for the PBs will be scaled to minimize the application's resource consumption while ensuring the end-to-end latency of microservices meets the SLO. Although abundant replicas can alleviate the performance degradation problem, they also consume a significant amount of resources. Consequently, it is essential to maintain a balance between performance guarantees and resource consumption. The process of *Scaling Decision* will be modeled as a constrained optimization problem to achieve this balance.

1) Constrained Optimization Model: The autoscaling optimization in our scenario seeks to identify an allocation schema that allocates a variable number of replicas for each PB. Given n PBs that require scaling, we define a strategy as a set $\mathbb{X} = \{x_1, x_2, \ldots, x_n\}$, where x_i denotes the number of replicas allocated for PB i. Before the optimization, the initial number of replicas for PBs can be expressed as $\mathbb{C} = \{c_1, c_2, \ldots, c_n\}$. It should be noted that the replicas constraint in PBScaler should be defined separately for scaled-down and scaled-up processes. During the scaled-up process, we limit the number of replicas for PBs as follows:

$$\begin{aligned} &x_i \geq c_i + 1, \forall x_i \in \mathbb{X}, \forall c_i \in \mathbb{C}, \\ &x_i \leq c^{\max}, \forall x_i \in \mathbb{X}, \end{aligned} \tag{7}$$

where $c^{\max}$ represents the maximum number of replicas that a microservice can scale to, given limited server resources. The constraint of the number of replicas during the scaled-down process can be expressed as

$$\begin{aligned} &x_i \geq \max(c_i - \gamma, 1), \forall x_i \in \mathbb{X}, \forall c_i \in \mathbb{C}, \\ &x_i \leq c_i, \forall x_i \in \mathbb{X}, \forall c_i \in \mathbb{C}. \end{aligned} \tag{8}$$

In (8), γ (with the default value of two) denotes the maximum number of replicas reductions. This limit is reasonable since reducing the number of microservice replicas drastically can cause a short latency peak, as observed in experiments.

The goal of the *Scaling Descision* is to minimize the application's resource consumption while maintaining its performance. Application performance is usually expressed by SLO violations that users are more concerned about. Therefore, the application performance reward can be detailed as

$$R_1 = \begin{cases} 0, & SLO\ violation, \\ 1, & w/o\ SLO\ violation. \end{cases} \tag{9}$$

During the optimization process, the application's resource consumption, such as CPU and memory usage, is unpredictable. To conservatively estimate resource consumption, we consider the ratio of PB replicas to the maximum number of allocatable replicas, rather than calculating the cost of CPU and memory. We calculate the resource reward as

$$R_2 = 1 - \frac{\sum_{i=1}^{n} x_i}{c^{\max} \times n}. \tag{10}$$

Our objective is to guarantee performance while minimizing resource consumption. We leverage a weighted linear combination (WLC) method to balance the two objectives. The final optimization objective is defined as

$$\max_{\mathbb{X}}(\lambda \cdot R_1(\mathbb{X}) + (1-\lambda) \cdot R_2(\mathbb{X})), \tag{11}$$

where $\lambda \in [0, 1]$. We set λ as a parameter to balance the application performance and resource consumption.

2) SLO Violation Predictor: To calculate the performance reward R_1, evaluating whether a strategy will cause the SLO violation in online applications is necessary. A simple way is to execute candidate strategies directly in online applications and wait for feedback from the monitoring system. However, oscillations caused by frequent scaling in online applications

Authorized licensed use limited to: University of Greenwich. Downloaded on April 07,2026 at 15:55:01 UTC from IEEE Xplore. Restrictions apply.

will be inevitable. An alternative method is to train an evaluation model with historical metric data, which can simulate the feedback from online applications. Without interacting with online applications, this model predicts application performance based on the current application state.

We use a vector $\boldsymbol{r}$ to denote the number of replicas for each microservice after executing the scaling strategy $\mathbb{X}$. $\boldsymbol{w}$ is a vector that denotes the current workload for each microservice. Because of the low time cost of bottleneck-aware optimization, it is reasonable to hypothesize that $\boldsymbol{w}$ will not change significantly during this period (see Section IV-B). Given the application state represented by workload $\boldsymbol{w}$ and replicas $\boldsymbol{r}$ of all microservices, an SLO violation predictor can be designed as

$$\psi(\boldsymbol{r}, \boldsymbol{w}) = \begin{cases} 0, & SLO\ violation, \\ 1, & w/o\ SLO\ violation, \end{cases} \tag{12}$$

where ψ is a binary classification model. Details of selecting an appropriate classification model will be discussed in Section IV-C. The historical metric data used for training can be generated using either a classical scaling method (the Kubernetes autoscaler by default) or a stochastic method. We deployed an open-source microservice system on three nodes (with a total of 44 CPU cores and 220 GB of RAM) and performed elastic scaling. Prometheus gathered each microservice's workload and P90 tail latency at regular time intervals. By comparing the tail latency of the front-end microservice with the SLO, monitoring data for each time interval can be easily labeled.

3) Autoscaling Optimization: As mentioned in Section II-I-C1, the tradeoff between performance and resource consumption can be modeled as a constrained optimization problem. To find a near-optimal strategy, PBScaler employs a genetic algorithm (GA) to generate and optimize scaling strategies that reduce resource consumption while meeting SLO requirements. By emulating natural selection in evolution, the GA improves the superior offspring while eliminating the inferior ones. Initially, the GA performs a random search to initialize a population of chromosomes in the strategy space, with each chromosome indicating a potential strategy for the optimization problem. Next, in each iteration, elite chromosomes with high fitness, called elites, will be selected for crossover or mutation to produce the next generation.

The autoscaling optimization in our scenario seeks to identify a scaling strategy that allocates a variable number of replicas for each PB. The process of autoscaling optimization is illustrated in Fig. 8. In the beginning, PBScaler obtains each microservice's current number of replicas $\boldsymbol{r}$ and workload $\boldsymbol{w}$. After the *Performance Bottleneck Analysis*, PBScaler identifies the PBs from $\boldsymbol{r}$ and filters out them to get $\boldsymbol{r'}$. Then, the population of strategies for PBs is generated by the Decision Maker. Since the number of microservices to be scaled influences the speed and effect of the optimization algorithm (Section IV-C), PBScaler assumes that only PBs need to be elastically scaled. In other words, the number of replicas in $\boldsymbol{r'}$ will remain unchanged. The SLO violation predictor is responsible for evaluating the generated strategies. It should be noted that the strategy is merged with $\boldsymbol{r'}$ and input to the SLO violation predictor together with $\boldsymbol{w}$. With the help of GA, the superior strategy $\mathbb{X}_{best}$ is selected and then merged with $\boldsymbol{r'}$ to generate the final decision.

Algorithm 3: GA-Based Autoscaling Optimization.

Input: the number of iterations I
the size of population N_p
the size of elites N_z
the probability of crossover p_c
the probability of mutation p_m
Output: the best scaling strategy s
1: $P_0 \leftarrow$ Randomly create a population with N_p strategies
2: Evaluate the fitness of each strategy in P_0
3: $Z_0 \leftarrow$ Get N_z elites from P_0
4: $s \leftarrow$ Get the best strategy in Z_0
5: $i \leftarrow 0$
6: **while** $i < I$ **do**
7: $F_i \leftarrow$ Set parents with $(N_p - N_z)$ strategies from P_i
8: $O_i \leftarrow$ Generate offspring by recombining F_i with a probability of p_c
9: $O_i \leftarrow$ Generate offspring by mutating O_i with a probability of p_m
10: $P_{i+1} \leftarrow (Z_i \cup O_i)$
11: Evaluate the fitness of each strategy in P_{i+1}
12: $Z_{i+1} \leftarrow$ Get N_z elites from P_{i+1}
13: $t \leftarrow$ Get the best strategy in Z_{i+1}
14: $f_s, f_t \leftarrow$ Evaluate the fitness of s and t
15: **if** $f_s < f_t$ **then**
16: $s \leftarrow t$
17: **end if**
18: $i \leftarrow i + 1$
19: **end while**

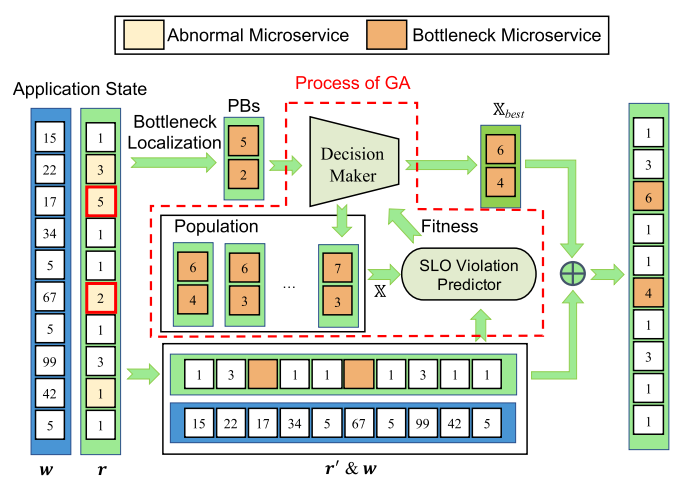


Fig. 8. Illustration of the autoscaling optimization process.

In the optimization phase, the Decision Maker generates and improves the scaling strategy for PB using the GA, as described in Algorithm 3. After randomly generating a population within the strategy scope of each PB (Line 1), the Decision Maker estimates the fitness of each strategy based on (11) and stores the elites (Lines 2-3). In each iteration, the GA uses a tournament-based selection operator to pick out outstanding parents F_i

Authorized licensed use limited to: University of Greenwich. Downloaded on April 07,2026 at 15:55:01 UTC from IEEE Xplore. Restrictions apply.

(Line 7). New offspring O_i are generated through recombination and mutation (Lines 8-9) using a two-point crossover operator and a binary-chromosome mutation operator. By simulating natural selection, new offspring O_i and elites Z_i with higher fitness constitute a new population P_{i+1} that enters the next iteration (Line 10). At each iteration, the current best strategy t undergoes a fitness comparison with the historically best strategy s (Lines 13–17). The best strategy s after I iterations will be considered as the value of final $\mathbb{X}_{best}$.

IV. Evaluations

In this section, we present the details of experimental scenarios for autoscaling, including a comparison of PBScaler with several state-of-the-art autoscaling algorithms from academia and industry.

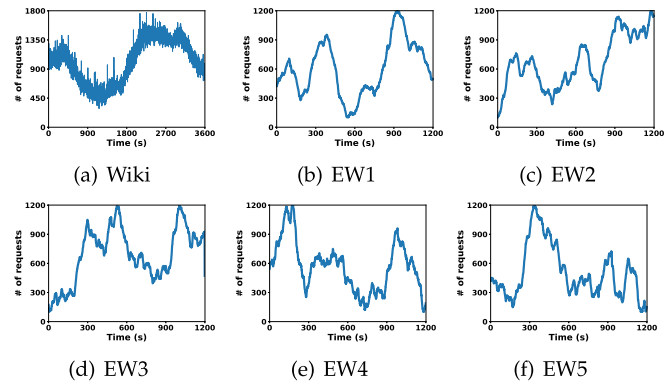
A. Experimental Setup

1) Microservice Platform: Experiments were carried out in our private cloud cluster, consisting of three physical computers (one master node and two worker nodes) with a total of 44 Intel 2.40 GHz CPU cores and 220 GB of RAM. To evaluate autoscaling, we selected two open-source microservice applications as benchmarks: a) Online Boutique,[7] a Web-based E-commerce demo application developed by Google. The system implements basic functions such as browsing products, adding items to shopping carts, and payment processing through the collaboration of ten stateless microservices and a Redis cache. b) Train Ticket:[8] a large-scale, open-source microservice system developed by Fudan University. With more than 40 microservices and the usage of MongoDB and MySQL for data storage, Train Ticket can satisfy a variety of workflows, such as online ticket browsing, booking, and purchasing. Because of cluster resource constraints, we limited each microservice to no more than eight replicas. The source code is available on Github.[9]

2) Workload: We evaluated the effectiveness of PBScaler under various traffic scenarios, using a real-world Wikipedia workload from the Wiki-Pageviews [40] on March 16, 2015, and five emulated workloads (EW1 $\sim$ EW5), inspired by the experiments conducted by Abdullah et al. in [41]. We compressed the real-world workload to one hour and scaled it to an appropriate level for our cluster. The five emulated workloads exhibited various patterns, such as single peak, multiple peaks, rising, and dropping, and were limited to a duration of twenty minutes. Fig. 9 depicts the fluctuation of these workloads.

3) Baseline Methods: We compare PBScaler with several state-of-the-art microservice autoscaling methods from academia and industry, which perform dynamic horizontal scaling of microservices from the perspectives of static thresholds, control theory, and black-box optimization.

- *Kubernetes Horizontal Pod Autoscaling (KHPA):* It is the default horizontal scaling scheme of Kubernetes. By customizing a threshold T for a certain resource R (CPU usage as the default) and aggregating the resource usage U_i^R from all replicas of a microservice, KHPA defines the target number of replicas as $n = \lceil \sum_{i \in ActivePods} U_i^R / T \rceil$.
- *MicroScaler [15]:* It is an autoscaling tool that uses a black-box optimization algorithm to determine the optimal number of replicas for a microservice. MicroScaler calculates the microservice's P90/P50 for classification and then performs four iterations of Bayesian Optimization to make a scaling decision.
- *SHOWAR [11]:* It is a hybrid autoscaling technology. We reproduced the horizontal scaling part in SHOWAR, which uses the PID control theory to gradually bring the observed metric close to the user-specified threshold. In our implementation, we replaced the run queue latency with the more common P90 latency since the former requires an additional eBPF tool.



(a) Wiki (b) EW1 (c) EW2 (d) EW3 (e) EW4 (f) EW5

Fig. 9. Fluctuation of one-hour real-world workloads (Wiki) and twenty minutes emulated workloads (EW).

4) Experimental Parameters and Evaluation Criteria: In our experiments, we fixed the collection interval of Prometheus to five seconds. With the increase in experiment time and workloads, the data volume required by stateful microservices like MongoDB will also grow. Eventually, the data volume will exceed the available memory, necessitating the use of disk storage. This transition can cause performance degradation that cannot be remedied through autoscaling. Hence, we limit workload testing to stateless traces. The SLO values for the Online Boutique and the Train Ticket were set to 500 ms and 200 ms, respectively. In the SLO violation detection and redundancy checking module, PBScaler first sets the action boundary α to 0.2 to reduce noise interference, as done in SHOWAR. Then, we empirically set the degree of significance β to 0.9 to control the workload level that triggers scaling. For bottleneck localization, the impact factor σ of the topological potential is set to 1, and the top-k ($k = 2$) microservices with the highest score in $\boldsymbol{rl}$ will be considered PBs.

We choose the SLO violation rate, resource consumption, and response time to evaluate the performance of autoscaling methods. An autoscaling approach is considered more effective if it can reduce response time, SLO violation rate, and resource consumption. We define the SLO violation rate as the percentage of the end-to-end P90 tail latency that exceeds the

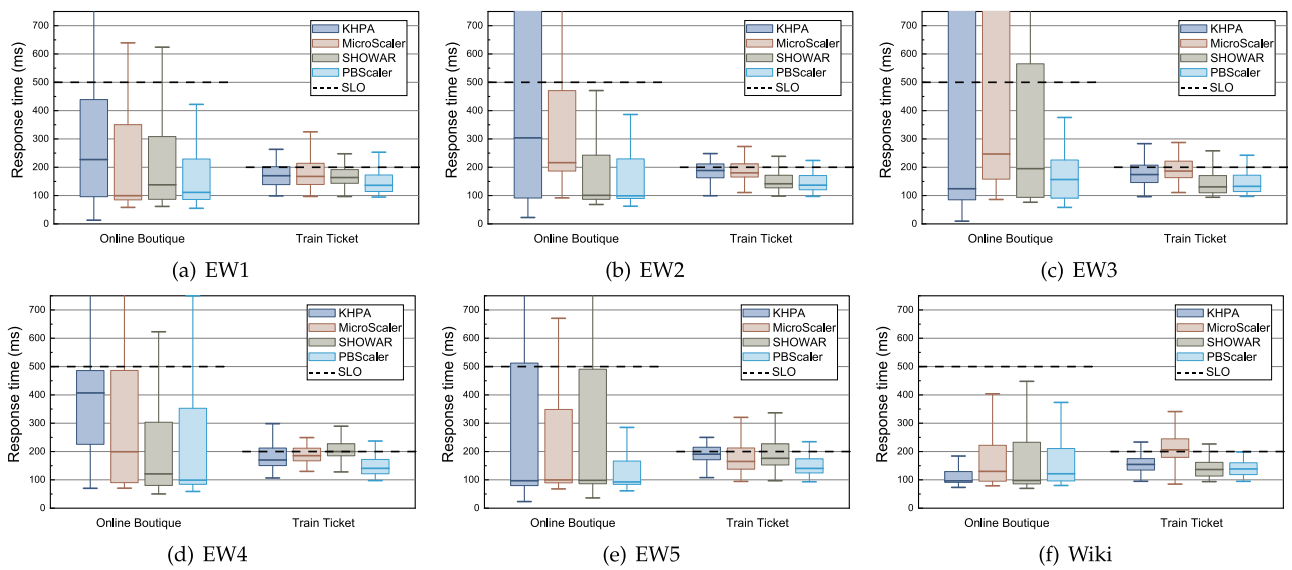
[7]https://github.com/GoogleCloudPlatform/microservices-demo
[8]https://github.com/FudanSELab/train-ticket
[9]https://github.com/WHU-AISE/PBScaler

Authorized licensed use limited to: University of Greenwich. Downloaded on April 07,2026 at 15:55:01 UTC from IEEE Xplore. Restrictions apply.

TABLE II
PERFORMANCE OF STATE-OF-THE-ART METHODS AND PBSCALER UNDER REAL AND EMULATED WORKLOADS

Methods	Online Boutique						Train Ticket					
	Wiki	EW1	EW2	EW3	EW4	EW5	Wiki	EW1	EW2	EW3	EW4	EW5
SLO violation rate (%)												
None	79.64	57.26	67.22	70.12	56.85	39.00	94.87	48.37	53.11	57.54	35.88	37.89
KHPA	**4.86**	13.45	37.08	31.95	20.00	25.10	10.48	37.50	40.09	30.46	29.72	30.90
MicroScaler	8.18	17.43	17.45	31.54	22.41	18.26	46.70	29.61	30.48	47.57	50.29	42.16
SHOWAR	13.74	13.33	**10.92**	27.39	12.08	25.00	9.39	20.14	22.92	19.23	25.66	21.04
PBScaler	5.69	**7.88**	12.45	**8.30**	**11.62**	**8.71**	**5.62**	**15.00**	**19.40**	**18.14**	**16.24**	**14.22**
Cost ($)												
None	1.93	0.50	0.62	0.65	0.61	0.64	11.62	3.03	3.06	3.25	3.15	3.39
KHPA	3.48	1.03	1.06	1.08	1.11	1.09	18.29	5.50	5.71	5.66	5.84	5.27
MicroScaler	3.09	0.79	0.72	**0.71**	0.76	**0.68**	15.44	3.76	3.52	4.70	4.86	4.46
SHOWAR	**2.49**	0.71	0.83	0.85	0.79	0.69	**13.64**	3.86	3.82	4.21	4.09	3.56
PBScaler	2.57	**0.68**	**0.69**	0.72	**0.63**	0.69	14.02	**3.57**	**3.30**	**3.64**	**3.69**	**3.14**

SLO violation rate and cost are reported.



(a) EW1 (b) EW2 (c) EW3 (d) EW4 (e) EW5 (f) Wiki

Fig. 10. Latency distribution for different methods under real and emulated workloads.

SLO. Resource consumption is calculated following the method presented in [42], where the CPU price is 0.00003334$(vCPU/s) and the memory price is 0.00001389$(G/s). The total resource consumption is obtained by summing the cost of memory and CPU.

B. Performance Evaluation

Table II compares the SLO violation rates and resource costs for the four autoscaling methods in two microservice applications with different workloads. The None method is used as a reference and performs no autoscaling operation. Its results are presented in grey and are excluded from the comparison.

In general, PBScaler outperforms the competing approaches in reducing SLO violations and minimizing resource overhead under six workloads in both microservices systems. In particular, the SLO violation rate of PBScaler in Train Ticket is, on average, 4.96% lower than that of the baseline methods, while the resource cost is reduced by an average of 0.24$. These results show that PBScaler can perform elastic scaling for bottleneck microservices in large-scale microservice systems quickly and precisely, thereby reducing SLO violations and saving resources. Regarding the six workloads in Online Boutique, PBScaler also achieves the lowest SLO violations in four of them and minimizes resource consumption in three emulated workloads.

Fig. 10 depicts the box plots of latency distribution for different methods under six workloads, exploring the impact of each method on the performance of the microservice system. It can be seen that the majority of the autoscaling methods can keep the median of the latency distribution below the red dotted line (SLO). However, only PBScaler goes a step further to reduce the third quartile significantly below the SLO for all workloads.

To evaluate the time cost of using PBScaler for elastic scaling, the average time required by each module in PBScaler is collected and counted. As reported in Table III, the total time cost of all PBScaler modules in Online Boutique is less than one monitoring interval (i.e., 5 s), while the same metric for Train Ticket is less than two monitoring intervals. Thanks to the PBA that narrows the decision-making scope, the time cost of the Decision Maker does not increase much (no more than 6.6%)

Authorized licensed use limited to: University of Greenwich. Downloaded on April 07,2026 at 15:55:01 UTC from IEEE Xplore. Restrictions apply.

TABLE III
TIME COST OF FOUR MODULES IN PBSCALER

Modules	Online Boutique	Train Ticket
SLO Violation Detection	0.29s	1.03s
Redundancy Checking	0.11s	0.15s
PBA	0.79s	3.10s
Decision Maker	3.36s	3.58s

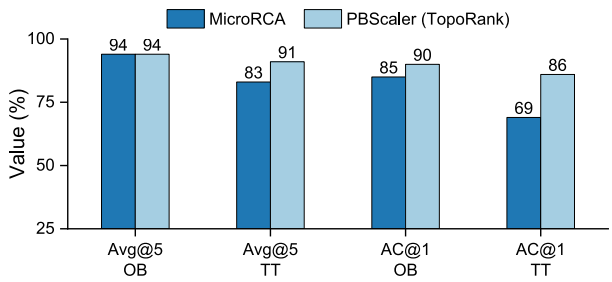


Fig. 11. Performance comparison of bottleneck localization on Online Boutique (OB) and Train Ticket (TT).

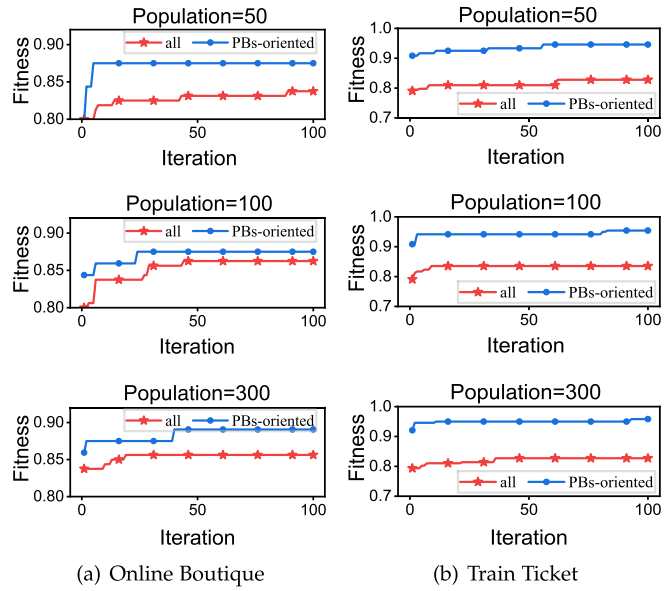


Fig. 12. Iterative process of the genetic algorithm under Online Boutique and Train Ticket.

when the application is switched from Online Boutique to Train Ticket, despite the increased number of microservices. However, we recognize the limitation that the time consumption of PBA quickly rises as the microservice scale grows, which will be our future work.

C. Effectiveness Analysis of Components

1) Performance Comparision of Bottleneck Localization: To evaluate whether the TopoRank algorithm can effectively locate PBs caused by burst workloads, we injected exceptions, such as CPU overload, memory overflow, and network congestion, into Online Boutique and Train Ticket through Chaos Mesh. These exceptions are typically caused by high-workload conditions. The TopoRank algorithm was used to analyze the metrics and identify performance bottlenecks for these exceptions. The localization results were then compared to MicroRCA [27], a baseline method for microservice root cause analysis. $AC@k$ measures the accuracy of the real PBs in the top-k results, and $Avg@k$ is the average accuracy at the top-k results. These metrics can be calculated as follows.

$$AC@k = \frac{1}{|A|} \sum_{a \in A} \frac{|RT@k \cap PBs|}{\min(k, |PBs|)}, \tag{13}$$

$$Avg@k = \frac{1}{|A|} \sum_{a \in A} \sum_{k=1}^{|A|} AC@k, \tag{14}$$

where A represents the set of exceptions, and $RT@k$ refers to the top-k microservices in the ranking list. Fig. 11 presents the $AC@1$ and $Avg@5$ values of TopoRank and MicroRCA across different microservice applications. The results indicate that TopoRank performs better than MicroRCA in both metrics. This is primarily due to the fact that TopoRank takes into account both the anomaly potential and microservice dependencies when performing Personalized PageRank.

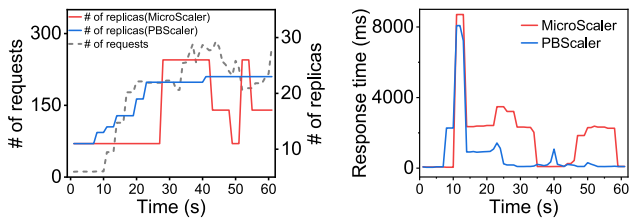
The primary purpose of bottleneck location is to narrow down the strategy space and expedite the discovery of the optimal strategy. We perform GA iterations on both PBs and all microservices to demonstrate the influence of bottleneck localization on optimization. Fig. 12 depicts the iterative process under the microservice systems and demonstrates that as the population increases, the PB-aware strategy significantly outperforms the approach that scales for all microservices in terms of fitness. The PB-aware strategy can obtain superior fitness in less than five iterations. In contrast, the all-microservices-involved method requires larger populations and more iterations to achieve the same level of fitness. This is attributed to the fact that the PB-aware strategy aids the genetic algorithm in reducing the optimization range precisely and accelerating the acquisition of superior solutions.

2) Effectiveness of the SLO Violation Predictor: The objective of the SLO violation predictor is to forecast the result of the optimization strategy directly rather than waiting for feedback from the online application. We determine whether performance issues will occur based on the number of replicas and workloads of each microservice. Selecting a suitable binary classification model for the prediction task is critical. With a data collection interval of five seconds, we collected two datasets, including a 3.1 k historical sampling dataset for Train Ticket and a 1.5 k dataset for Online Boutique, in our cluster. For training and testing on these two datasets, we adopt four classical machine learning (ML) methods, including Support Vector Machine (SVM), Random Forest, Multilayer Perceptron (MLP), and Decision Tree. We employ typical binary classification indicators to assess the performance of experimental methods: $Precision = \frac{TP}{TP+FP}$ and $Recall = \frac{TP}{TP+FN}$, where TP represents the number of identified SLO violations; FN and FP represent the number of unrecognized SLO violations and the number of false-positive SLO violations, respectively. Table IV shows the performance comparison of the four methods for SLO violation prediction.

Authorized licensed use limited to: University of Greenwich. Downloaded on April 07,2026 at 15:55:01 UTC from IEEE Xplore. Restrictions apply.

TABLE IV
PRECISION AND RECALL OF FOUR ML METHODS FOR SLO VIOLATION PREDICTION

Method	Train Ticket		Online Boutique	
	Precision	*Recall*	*Precision*	*Recall*
SVM	0.819	0.961	0.865	0.915
Decision Tree	0.891	0.918	0.927	0.941
Random Forest	**0.919**	**0.963**	**0.956**	**0.969**
MLP	0.799	0.930	0.831	0.907



(a) Replicas fluctuation (b) Latency fluctuation

Fig. 13. Replica and latency fluctuation of PBScaler and MicroScaler under the Wiki workload.

According to the effects of the two datasets, we finally choose Random Forest as the primary algorithm for the SLO violation predictor.

To demonstrate that the SLO violation predictor can substitute the feedback from the real environment, we compare PBScaler, which employs the SLO violation predictor, with MicroScaler, which collects feedback from the online system. We injected burst workloads into the Online Boutique and made only one microservice abnormal to eliminate the difference in bottleneck localization between the two methods. As shown in Fig. 13, with the guidance of the predictor, the number and the frequency of decision-making attempts made by PBScaler are much lower than those of MicroScaler. Reducing online attempts in a cluster will evidently reduce the risk of oscillations.

V. CONCLUSION

This paper presents PBScaler, a bottleneck-aware autoscaling framework designed to prevent performance degeneration in microservice-based applications. PBScaler collects real-time performance metrics of applications using the service mesh technology and dynamically builds a correlation graph among microservices. To handle abnormal microservices caused by external dynamic workloads and intricate invocations among microservices, PBScaler employs TopoRank, a random walk algorithm based on the topological potential theory, to identify bottleneck microservices. Furthermore, PBScaler performs an offline evolutionary algorithm to optimize scaling strategies guided by an SLO violation predictor. Experimental results indicate that PBScaler can minimize resource consumption while achieving lower SLO violations.

In the future, we plan to improve our work from the following two aspects. First, we will explore the potential of using bottleneck awareness in finer-grained resource (e.g., CPU and memory) management. Second, we will explore how to circumvent the interference of stateful microservices in autoscaling since the performance degradation from stateful microservices may disrupt the autoscaling controller. Third, we will improve the efficiency of performance bottleneck analysis for large-scale microservice systems.

REFERENCES

[1] M. Ma, J. Xu, Y. Wang, P. Chen, Z. Zhang, and P. Wang, "AutoMAP: Diagnose your microservice-based web applications automatically," in *Proc. Web Conf.*, 2020, pp. 246–258.
[2] M. Kim, R. Sumbaly, and S. Shah, "Root cause detection in a service-oriented architecture," *ACM SIGMETRICS Perform. Eval. Rev.*, vol. 41, no. 1, pp. 93–104, 2013.
[3] H. Shan et al., "ϵ-diagnosis: Unsupervised and real-time diagnosis of small-window long-tail latency in large-scale microservice platforms," in *Proc. World Wide Web Conf.*, 2019, pp. 3215–3222.
[4] H. Qiu, S. S. Banerjee, S. Jha, Z. T. Kalbarczyk, and R. K. Iyer, "FIRM: An intelligent fine-grained resource management framework for SLO-Oriented microservices," in *Proc. 14th USENIX Symp. Operating Syst. Des. Implementation*, 2020, pp. 805–825.
[5] B. Liu, R. Buyya, and A. Nadjaran Toosi, "A fuzzy-based auto-scaler for web applications in cloud computing environments," in *Proc. Int. Conf. Service-Oriented Comput.*, Springer, 2018, pp. 797–811.
[6] S. Zhang et al., "Robust system instance clustering for large-scale web services," in *Proc. ACM Web Conf.*, 2022, pp. 1785–1796.
[7] J. Lin, P. Chen, and Z. Zheng, "Microscope: Pinpoint performance issues with causal graphs in micro-service environments," in *Proc. Int. Conf. Service-Oriented Comput.*, Springer, 2018, pp. 3–20.
[8] D. Liu et al., "MicroHECL: High-efficient root cause localization in large-scale microservice systems," in *Proc. IEEE/ACM 43rd Int. Conf. Softw. Eng.: Softw. Eng. Pract.*, 2021, pp. 338–347.
[9] B. Choi, J. Park, C. Lee, and D. Han, "pHPA: A proactive autoscaling framework for microservice chain," in *Proc. 5th Asia-Pacific Workshop Netw.*, 2021, pp. 65–71.
[10] A. Kwan, J. Wong, H.-A. Jacobsen, and V. Muthusamy, "HyScale: Hybrid and network scaling of dockerized microservices in cloud data centres," in *Proc. IEEE 39th Int. Conf. Distrib. Comput. Syst.*, 2019, pp. 80–90.
[11] A. F. Baarzi and G. Kesidis, "SHOWAR: Right-sizing and efficient scheduling of microservices," in *Proc. ACM Symp. Cloud Comput.*, 2021, pp. 427–441.
[12] J. Park, B. Choi, C. Lee, and D. Han, "GRAF: A graph neural network based proactive resource allocation framework for SLO-oriented microservices," in *Proc. 17th Int. Conf. Emerg. Netw. EXperiments Technol.*, 2021, pp. 154–167.
[13] Y. Al-Dhuraibi, F. Paraiso, N. Djarallah, and P. Merle, "Elasticity in cloud computing: State of the art and research challenges," *IEEE Trans. Services Comput.*, vol. 11, no. 2, pp. 430–447, Mar./Apr. 2018.
[14] P. Chen, Y. Qi, and D. Hou, "CauseInfer: Automated end-to-end performance diagnosis with hierarchical causality graph in cloud environment," *IEEE Trans. Serv. Comput.*, vol. 12, no. 2, pp. 214–230, Mar./Apr. 2019.
[15] G. Yu, P. Chen, and Z. Zheng, "Microscaler: Automatic scaling for microservices with an online learning approach," in *Proc. IEEE Int. Conf. Web Serv.*, 2019, pp. 68–75.
[16] U. Sharma, P. Shenoy, S. Sahu, and A. Shaikh, "A cost-aware elasticity provisioning system for the cloud," in *Proc. 31st Int. Conf. Distrib. Comput. Syst.*, 2011, pp. 559–570.
[17] R. da Rosa Righi, V. F. Rodrigues, C. A. Da Costa, G. Galante, L. C. E. De Bona, and T. Ferreto, "AutoElastic: Automatic resource elasticity for high performance applications in the cloud," *IEEE Trans. Cloud Comput.*, vol. 4, no. 1, pp. 6–19, First Quarter 2016.
[18] N. Roy, A. Dubey, and A. Gokhale, "Efficient autoscaling in the cloud using predictive models for workload forecasting," in *Proc. IEEE 4th Int. Conf. Cloud Comput.*, 2011, pp. 500–507.
[19] J. Wang, G. Zhao, H. Xu, Y. Zhao, X. Yang, and H. Huang, "TRUST: Real-time request updating with elastic resource provisioning in clouds," in *Proc. IEEE Conf. Comput. Commun.*, 2022, pp. 620–629.
[20] T. Jia, P. Chen, L. Yang, Y. Li, F. Meng, and J. Xu, "An approach for anomaly diagnosis based on hybrid graph model with logs for distributed services," in *Proc. IEEE Int. Conf. Web Serv.*, 2017, pp. 25–32.
[21] A. Nandi, A. Mandal, S. Atreja, G. B. Dasgupta, and S. Bhattacharya, "Anomaly detection using program control flow graph mining from execution logs," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2016, pp. 215–224.

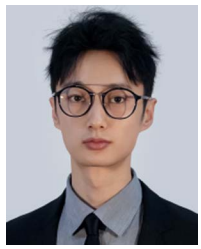
Authorized licensed use limited to: University of Greenwich. Downloaded on April 07,2026 at 15:55:01 UTC from IEEE Xplore. Restrictions apply.

[22] G. Yu et al., "MicroRank: End-to-end latency issue localization with extended spectrum analysis in microservice environments," in *Proc. Web Conf.*, 2021, pp. 3087–3098.
[23] G. Yu, Z. Huang, and P. Chen, "TraceRank: Abnormal service localization with dis-aggregated end-to-end tracing data in cloud native systems," *J. Softw. Evol. Process*, vol. 35, 2021, Art. no. e2413.
[24] H. Mi, H. Wang, Y. Zhou, M. R.-T. Lyu, and H. Cai, "Toward fine-grained, unsupervised, scalable performance diagnosis for production cloud computing systems," *IEEE Trans. Parallel Distrib. Syst.*, vol. 24, no. 6, pp. 1245–1255, Jun. 2013.
[25] Z. Li et al., "Practical root cause localization for microservice systems via trace analysis," in *Proc. IEEE/ACM 29th Int. Symp. Qual. Service*, 2021, pp. 1–10.
[26] Z. Zhang, B. Li, J. Wang, and Y. Liu, "AAMR: Automated anomalous microservice ranking in cloud-native environment," in *Proc. Int. Conf. Softw. Eng. Knowl. Eng.*, 2021, pp. 86–91.
[27] L. Wu, J. Tordsson, E. Elmroth, and O. Kao, "MicroRCA: Root cause localization of performance issues in microservices," in *Proc. IEEE/IFIP Netw. Operations Manage. Symp.*, 2020, pp. 1–9.
[28] Y. Meng et al., "Localizing failure root causes in a microservice through causality inference," in *Proc. IEEE/ACM 28th Int. Symp. Qual. Service*, 2020, pp. 1–10.
[29] D. Balla, C. Simon, and M. Maliosz, "Adaptive scaling of kubernetes pods," in *Proc. IEEE/IFIP Netw. Operations Manage. Symp.*, 2020, pp. 1–5.
[30] E. Casalicchio and V. Perciballi, "Auto-scaling of containers: The impact of relative and absolute metrics," in *Proc. IEEE 2nd Int. Workshops Found. Appl. Self* Syst.*, 2017, pp. 207–214.
[31] M. R. Hossen and M. A. Islam, "A lightweight workload-aware microservices autoscaling with QoS assurance," 2022, *arXiv:2202.00057*.
[32] L. Baresi and G. Quattrocchi, "A simulation-based comparison between industrial autoscaling solutions and COCOs for cloud applications," in *Proc. IEEE Int. Conf. Web Serv.*, 2020, pp. 94–101.
[33] J. Tong, M. Wei, M. Pan, and Y. Yu, "A holistic auto-scaling algorithm for multi-service applications based on balanced queuing network," in *Proc. IEEE Int. Conf. Web Serv.*, 2021, pp. 531–540.
[34] A. U. Gias, G. Casale, and M. Woodside, "ATOM: Model-driven autoscaling for microservices," in *Proc. IEEE 39th Int. Conf. Distrib. Comput. Syst.*, 2019, pp. 1994–2004.
[35] Z. Yang, P. Nguyen, H. Jin, and K. Nahrstedt, "MIRAS: Model-based reinforcement learning for microservice resource allocation over scientific workflows," in *Proc. IEEE 39th Int. Conf. Distrib. Comput. Syst.*, 2019, pp. 122–132.
[36] X. Zhou et al., "Latent error prediction and fault localization for microservice applications by learning from system trace logs," in *Proc. 27th ACM Joint Meeting Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng.*, 2019, pp. 683–694.
[37] J. Hu, Y. Han, and J. Hu, "Topological potential: Modeling node importance with activity and local effect in complex networks," in *Proc. 2nd Int. Conf. Comput. Model. Simul.*, 2010, pp. 411–415.
[38] Y. Mao, L. Zhou, and N. Xiong, "TPS: A topological potential scheme to predict influential network nodes for intelligent communication in social networks," *IEEE Trans. Netw. Sci. Eng.*, vol. 8, no. 1, pp. 529–540, First Quarter 2021.
[39] G. Jeh and J. Widom, "Scaling personalized web search," in *Proc. 12th Int. Conf. World Wide Web*, 2003, pp. 271–279.
[40] O. Keyes, "Wiki-pageviews, English wikipedia pageviews by second," 2015. [Online]. Available: http://datahub.io/dataset/english-wikipedia-pageviews-by-second
[41] M. Abdullah, W. Iqbal, J. L. Berral, J. Polo, and D. Carrera, "Burst-aware predictive autoscaling for containerized microservices," *IEEE Trans. Services Comput.*, vol. 15, no. 3, pp. 1448–1460, May/Jun. 2022.
[42] Z. Ding and Q. Huang, "COPA: A combined autoscaling method for kubernetes," in *Proc. IEEE Int. Conf. Web Serv.*, 2021, pp. 416–425.

Shuaiyu Xie received the BS degree from the School of Computer Science, Wuhan University, China, in 2021. He is currently working toward the PhD degree with Wuhan University. His current research interests include services computing, artificial intelligence for IT operations (AIOps), and task scheduling.

Jian Wang (Member, IEEE) received the PhD degree in computer science from Wuhan University, China, in 2008. He is currently an associate professor with the School of Computer Science, Wuhan University, China. His current research interests include services computing and software engineering. He is now a senior member of the China Computer Federation (CCF), and a member of the CCF Technical Committee on Services Computing (TCSC).

Bing Li (Member, IEEE) received the PhD degree from the Huazhong University of Science and Technology, Wuhan, China, in 2003. He is currently a professor with the School of Computer Science, Wuhan University. His main research areas are services computing, software engineering, artificial intelligence, and cloud computing. He is now a distinguished member of the China Computer Federation (CCF).



Zekun Zhang received the MS degree from the School of Computer Science, Wuhan University, China. He is currently working toward the PhD degree with the School of Computer Science, Wuhan University. His research interests include cloud computing and fault diagnosis.

Duantengchuan Li is currently working toward the PhD degree in software engineering with the School of Computer Science, Wuhan University, Wuhan, China. His research interests include recommendation system, representation learning, natural language processing, intention recognition, pattern recognition, computer vision and their applications in software engineering, intelligent education, and intelligent sports.

Patrick C. K. Hung received the bachelor's degree in computer science from the University of New South Wales, Sydney, NSW, Australia, in 1993, the master's and PhD degrees in computer science from the Hong Kong University of Science and Technology, Hong Kong, in 1995 and 2001, respectively, and the master's degree in management sciences from the University of Waterloo, Canada, in 2002. He is a professor and the director of International Programs with the Faculty of Business and Information Technology, Ontario Tech University, Canada. His research interests include services computing, cloud computing, Big Data, and edge computing. He is a founding member of the IEEE Technical Committee on Services Computing and the *IEEE Transactions on Services Computing*.

Authorized licensed use limited to: University of Greenwich. Downloaded on April 07,2026 at 15:55:01 UTC from IEEE Xplore. Restrictions apply.